

Porject Report

In this task I will be creating a machine learning model to successfully predict the status of a cirrhosis patient. The data set includes all the necessary features required to create the model. The dataset is of size (312, 20).

	N_Days	Status	Drug	Age	Sex	Ascites	Hepatomegaly	Spiders	Edema	Bilirubin	Cholesterol	Albumin	Copper	Alk_Phos	SGOT	Tryglicerides	Platelets
0	400	D	D-penicillamine	21464	F	Y	Y	Y	Y	14.5	261.0	2.60	156.0	1718.0	137.95	172.0	190
1	4500	C	D-penicillamine	20617	F	N	Y	Y	N	1.1	302.0	4.14	54.0	7394.8	113.52	88.0	221
2	1012	D	D-penicillamine	25594	M	N	N	N	S	1.4	176.0	3.48	210.0	516.0	96.10	55.0	151
3	1925	D	D-penicillamine	19994	F	N	Y	Y	S	1.8	244.0	2.54	64.0	6121.8	60.63	92.0	183
4	1504	CL	Placebo	13918	F	N	Y	Y	N	3.4	279.0	3.53	143.0	671.0	113.15	72.0	136

Part 1

In order to create the models, I had to break the dataset into training and testing set. I used `test_train_split` from `sklearn` to split the data with the test size of 28%.

```
[156] X_train.shape
```

```
⇒ (224, 18)
```

```
[157] X_test.shape
```

```
⇒ (88, 18)
```

The feature types of `X_train` and `X_test` are shown below.

X Training Data Feature Types:	X Test Data Feature Types:
N_Days int64	N_Days int64
Drug object	Drug object
Age int64	Age int64
Sex object	Sex object
Ascites object	Ascites object
Hepatomegaly object	Hepatomegaly object
Spiders object	Spiders object
Edema object	Edema object
Bilirubin float64	Bilirubin float64
Cholesterol float64	Cholesterol float64
Albumin float64	Albumin float64
Copper float64	Copper float64
Alk_Phos float64	Alk_Phos float64
SGOT float64	SGOT float64
Tryglicerides float64	Tryglicerides float64
Platelets float64	Platelets float64
Prothrombin float64	Prothrombin float64
Stage float64	Stage float64
dtype: object	dtype: object

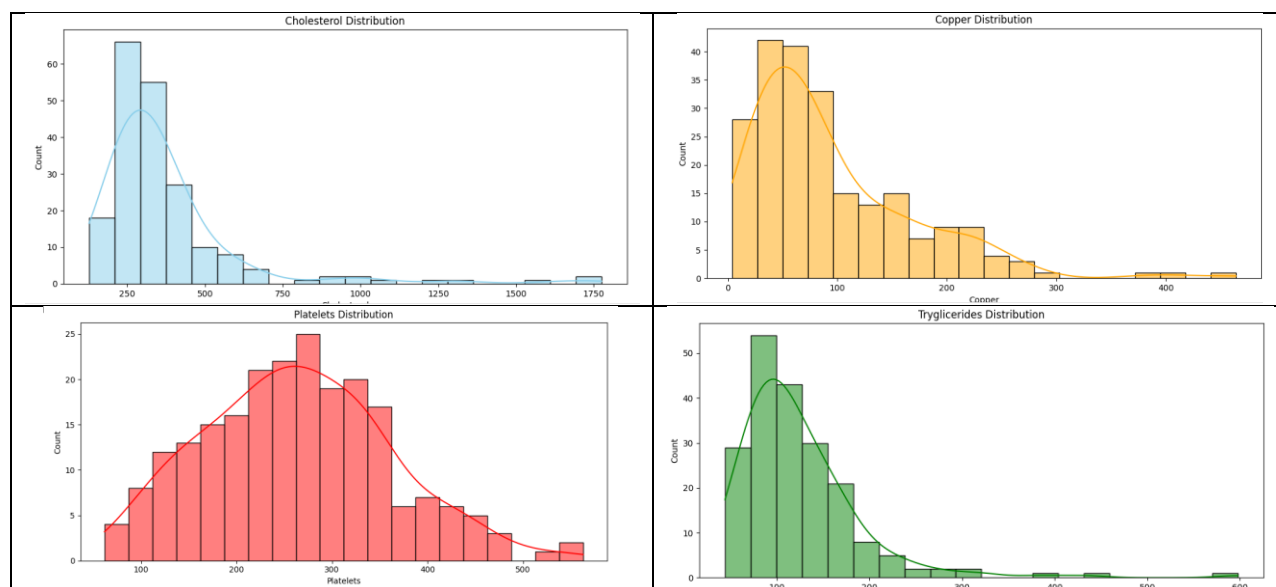
The training and testing data set had some missing values as well which are shown below.

```
[159] # Identify features with missing values
print("Features with Missing Values in X Training Data:")
print(X_train.columns[X_train.isnull().any()])
print("\nFeatures with Missing Values in X Test Data:")
print(X_test.columns[X_test.isnull().any()])
```

→ Features with Missing Values in X Training Data:
Index(['Cholesterol', 'Copper', 'Tryglicerides', 'Platelets'], dtype='object')

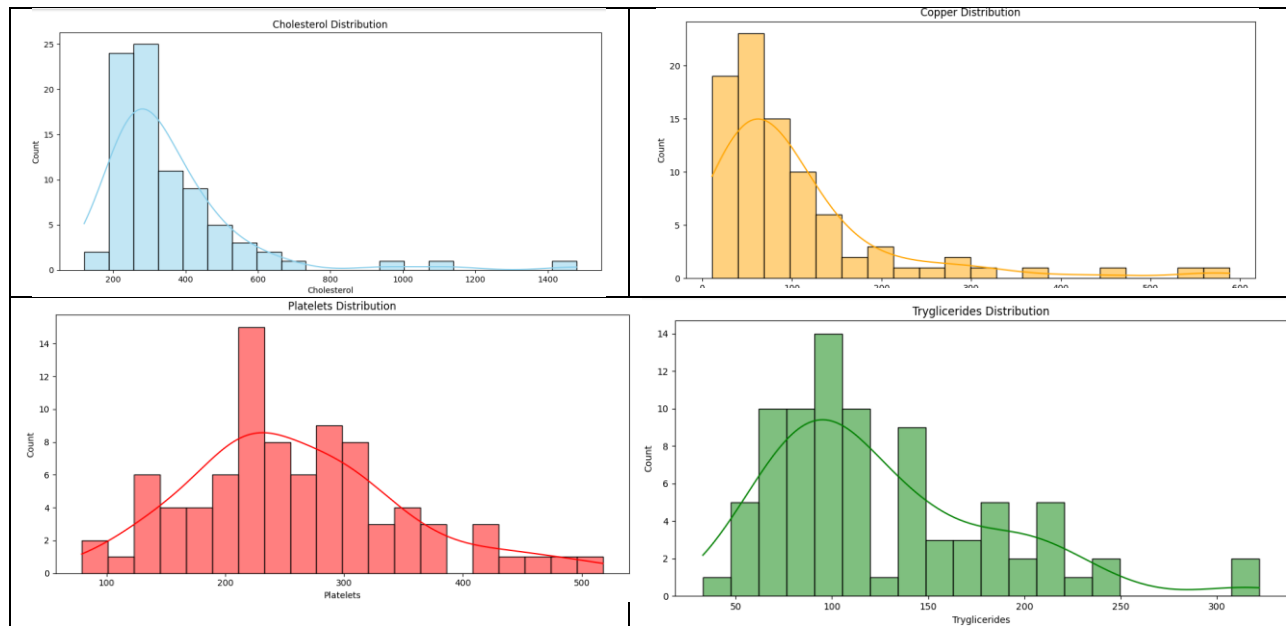
Features with Missing Values in X Test Data:
Index(['Cholesterol', 'Copper', 'Tryglicerides', 'Platelets'], dtype='object')

In order to fill these missing values I had to check whether the distribution was symmetric or skewed. For that purpose I plot their distribution curves to check their distribution. Below are the distribution curves of training set.



Since mean is greater than median for all the columns, the distribution is right-skewed, hence replacing missing values by median is a better approach.

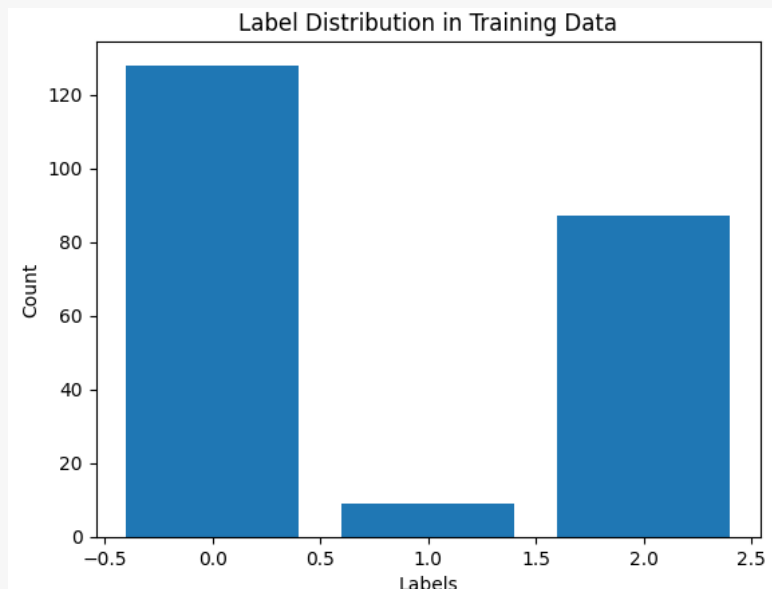
I applied the similar process for the training set as well.



Since the data is skewed so I replaced the missing values with median.

The categorical variables in the training and testing dataset included, 'Drug', 'Sex', 'Ascites', 'Hepatomegaly', 'Spiders', 'Edema' and 'Status' itself. I used Label encoder to encode these labels because the model needs numeric values to work properly.

The label distribution in the training data was not balanced at all; I drew a histogram plot to estimate the counts of each label in the training data.



The labels with value 0 were very high compared to the labels with value 1, the labels with value 2 were moderate but a lot higher than those of 1, hence the distribution of these labels were not balance and it caused a lot of issues with model training.

Part 2

Iteration 1

With the training data in mind, I decided to make 3 machine learning models, Logistic regression, Random forest and Gradient boosting. Logistic regression is computationally efficient and works well with large datasets, Random Forest builds multiple decision trees and combines their predictions, which reduces over fitting compared to individual decision trees and Gradient Boosting typically produces high predictive accuracy by iteratively improving upon the weaknesses of previous models.

- I used techniques like cross validation to train the models properly.
- I also used grid search to extract the best possible hyper parameters for every model.
- Since the class labels were not balanced so I used Ada boost to adjust the weights of the labels so that the model can learn the weak learners
- I also used bagging to get the best possible model in the end.

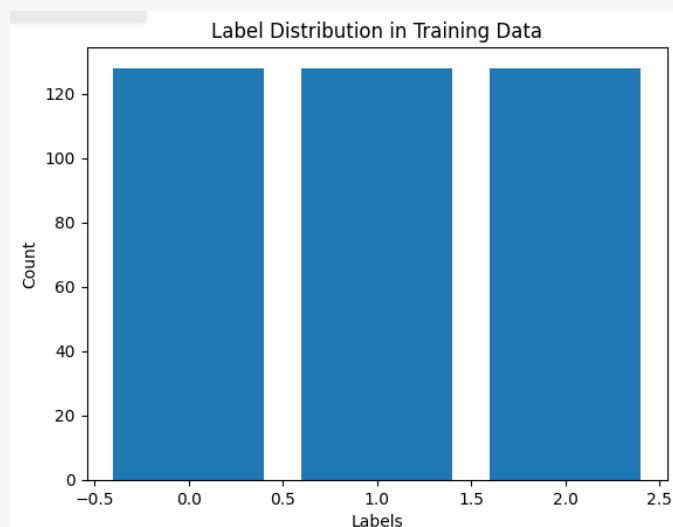
However, with all my tries, the labels which were very less in the dataset (1), were becoming hard to learn by the model. I came to a conclusion that I have to increase the number of samples of that specific label in the training set so that model can learn properly.

Iteration 2

I decided to use SMOTE algorithm to generate synthetic labels in the training dataset, so that the label imbalance issue can be resolved.

```
from imblearn.over_sampling import SMOTE
smote = SMOTE(sampling_strategy='auto', random_state=42)

# Resample data
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```



After the imbalance issue was resolved, I trained the three classifiers using cross validation with 5 folds.

- Logistic regression

```
from sklearn.model_selection import cross_val_predict
from sklearn.metrics import classification_report
# Train and evaluate models using cross-validation

# Perform cross-validation predictions
y_pred_log = cross_val_predict(logistic_regression, X_resampled, y_resampled, cv=5)
# Generate classification report
report = classification_report(y_resampled, y_pred_log)
print("Logistic Regression")
print(report)
```

Logistic Regression					
	precision	recall	f1-score	support	
0	0.76	0.76	0.76	128	
1	0.81	0.93	0.87	128	
2	0.81	0.70	0.75	128	
accuracy			0.79	384	
macro avg	0.79	0.79	0.79	384	
weighted avg	0.79	0.79	0.79	384	

- Random forest

```
#random Forst
y_pred_random = cross_val_predict(random_forest, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_random)
print("Random Forest")
print(report)
```

Random Forest					
	precision	recall	f1-score	support	
0	0.84	0.84	0.84	128	
1	0.94	0.99	0.97	128	
2	0.85	0.80	0.83	128	
accuracy			0.88	384	
macro avg	0.88	0.88	0.88	384	
weighted avg	0.88	0.88	0.88	384	

- Gradient Boosting

```
[ ] #Gradient Boosting
y_pred_gb = cross_val_predict(gradient_boosting, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_gb)
print("Gradient Boosting")
print(report)
```

Gradient Boosting					
	precision	recall	f1-score	support	
0	0.85	0.81	0.83	128	
1	0.91	0.98	0.95	128	
2	0.84	0.80	0.82	128	
accuracy			0.87	384	
macro avg	0.87	0.87	0.87	384	
weighted avg	0.87	0.87	0.87	384	

The training set looks to be giving good results but I had to make sure that each model is given correct hyper parameters so that they can give good result for the testing set. For that reason I took help of the GridSearchCV to identify the best possible hyper parameters for each of the models.

```
from sklearn.model_selection import GridSearchCV

# Hyperparameter grid for each model
param_grid_logistic = {'C': [0.1, 1, 10, 100]}
param_grid_rf = {'n_estimators': [50, 100, 200], 'max_depth': [None, 10, 20]}
param_grid_gb = {
    'n_estimators': [50, 100, 200],
    'learning_rate': [0.01, 0.1, 0.2],
    'max_depth': [3, 5, 7]
}

# Grid search for hyperparameter tuning
grid_search_logistic = GridSearchCV(LogisticRegression(max_iter=1000), param_grid_logistic, cv=5)
grid_search_rf = GridSearchCV(RandomForestClassifier(), param_grid_rf, cv=5)
#grid_search_svm = GridSearchCV(SVC(), param_grid_svm, cv=5)
grid_search_gb = GridSearchCV(GradientBoostingClassifier(), param_grid_gb, cv=5)

# Fit grid search models
grid_search_logistic.fit(X_resampled, y_resampled)
grid_search_rf.fit(X_resampled, y_resampled)
#grid_search_svm.fit(X_resampled, y_resampled)
grid_search_gb.fit(X_resampled, y_resampled)

# Get best hyperparameters
best_params_logistic = grid_search_logistic.best_params_
best_params_rf = grid_search_rf.best_params_
#best_params_svm = grid_search_svm.best_params_
best_params_gb = grid_search_gb.best_params_

print("Best Hyperparameters for Logistic Regression:", best_params_logistic)
print("Best Hyperparameters for Random Forest:", best_params_rf)
#print("Best Hyperparameters for SVM:", best_params_svm)
print("Best hyperparameters for Gradient Boosting:", best_params_gb)
```

The hyper parameters that I choose for the logistic regression, it is C, which controls the regularization strength to reduce over fitting. For random forest, the parameters are n_estimators and max_depth which tells the number of trees and the maximum depth of each tree. For gradient boosting the n_estimators tells the number of boosting stages, learning_rate scales the contribution of each weak learner in the gradient boosting ensemble and max_depth controls the maximum depth of each tree.

The parameters that I got are as follows

- LogisticRegression(C=100, max_iter=1000)
- RandomForestClassifier(max_depth = 5, n_estimators = 100)
- GradientBoostingClassifier(learning_rate = 0.1, max_depth = 3, n_estimators = 50)

These are the hyper parameters I got after a number of trials.

After getting the best hyper parameters, I used the ada boost to get adjust the weights of each model.

- Logistic regression

```
#Using hyper parameters
logistic_regression = LogisticRegression(C=100, max_iter=1000)
random_forest = RandomForestClassifier(max_depth = 5, n_estimators = 100)
#svm_classifier = SVC(C=100, gamma='auto')
gradient_boosting = GradientBoostingClassifier(learning_rate = 0.1, max_depth = 3, n_estimators = 50)

# Using Adaboost to adjust weights of the Logistic regression model
ada_clf_lr = AdaBoostClassifier(estimator=logistic_regression, n_estimators=50, learning_rate=1.0, random_state=42)
y_pred_ada_lr = cross_val_predict(ada_clf_lr, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_ada_lr)
print("Logistic Regression with Ada boost")
print(report)
print()
```

Logistic Regression with Ada boost				
	precision	recall	f1-score	support
0	0.68	0.66	0.67	128
1	0.80	0.90	0.85	128
2	0.73	0.67	0.70	128
accuracy			0.74	384
macro avg	0.74	0.74	0.74	384
weighted avg	0.74	0.74	0.74	384

- Random Forest

```
#Random forest
ada_clf_rf = AdaBoostClassifier(estimator=random_forest, n_estimators=50, learning_rate=1.0, random_state=42)
y_pred_ada_rf = cross_val_predict(ada_clf_rf, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_ada_rf)
print("Random Forest with Ada boost")
print(report)
print()
```

Random Forest with Ada boost				
	precision	recall	f1-score	support
0	0.84	0.83	0.83	128
1	0.93	1.00	0.97	128
2	0.86	0.81	0.84	128
accuracy			0.88	384
macro avg	0.88	0.88	0.88	384
weighted avg	0.88	0.88	0.88	384

- Gradient boosting

```
#Gradient Boosting
ada_clf_gb = AdaBoostClassifier(estimator=gradient_boosting, n_estimators=50, learning_rate=1.0, random_state=42)
y_pred_ada_gb = cross_val_predict(ada_clf_gb, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_ada_gb)
print("Gradient boosting with Ada boost")
print(report)
```

Gradient boosting with Ada boost				
	precision	recall	f1-score	support
0	0.84	0.81	0.83	128
1	0.91	0.99	0.95	128
2	0.85	0.80	0.83	128
accuracy			0.87	384
macro avg	0.87	0.87	0.87	384
weighted avg	0.87	0.87	0.87	384

After the getting the weights of the models right, I decided to use another ensemble technique, bagging, to get the best performing model before testing the model on test set.

- Logistic Regression

```
# Define Bagging classifier to get the best model
bagging_clf_lr = BaggingClassifier(estimator=ada_clf_lr, n_estimators=50, random_state=42)
y_pred_bagging_lr = cross_val_predict(bagging_clf_lr, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_bagging_lr)
print("Logistic Regression with bagging")
print(report)
```

Logistic Regression with bagging				
	precision	recall	f1-score	support
0	0.76	0.73	0.74	128
1	0.82	0.97	0.89	128
2	0.79	0.69	0.74	128
accuracy			0.79	384
macro avg	0.79	0.79	0.79	384
weighted avg	0.79	0.79	0.79	384

- Random Forest

```
bagging_clf_rf = BaggingClassifier(estimator=ada_clf_rf, n_estimators=50, random_state=42)
y_pred_bagging_rf = cross_val_predict(bagging_clf_rf, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_bagging_rf)
print("Random Forest with bagging")
print(report)
```

Random Forest with bagging				
	precision	recall	f1-score	support
0	0.84	0.82	0.83	128
1	0.89	1.00	0.94	128
2	0.86	0.77	0.81	128
accuracy			0.86	384
macro avg	0.86	0.86	0.86	384
weighted avg	0.86	0.86	0.86	384

- Gradient Boosting

```
bagging_clf_gb = BaggingClassifier(estimator=ada_clf_gb, n_estimators=50, random_state=42)
y_pred_bagging_gb = cross_val_predict(bagging_clf_gb, X_resampled, y_resampled, cv=5)
report = classification_report(y_resampled, y_pred_bagging_gb)
print("Gradient boosting with bagging")
print(report)
```

Gradient boosting with bagging				
	precision	recall	f1-score	support
0	0.84	0.80	0.82	128
1	0.91	1.00	0.95	128
2	0.84	0.79	0.81	128
accuracy			0.86	384
macro avg	0.86	0.86	0.86	384
weighted avg	0.86	0.86	0.86	384

After performing all the techniques, the models seemed to be doing alright for the class labels 0 and 2, however for the class label 1, the models seemed to be a bit over fit. The model that I thought worked

best was the random forest one as it showed the best results for all the classes, in terms of precision, recall and f1 score.

Test set results

```
#Random forest test
bagging_clf_rf.fit(X_resampled, y_resampled)
y_pred_random = bagging_clf_rf.predict(X_test)
print("Random Forest Test Set Performance:")
print(classification_report(y_test, y_pred_random))
```

```
Random Forest Test Set Performance:
              precision    recall  f1-score   support

     0           0.73       0.88       0.80         40
     1           0.67       0.20       0.31         10
     2           0.78       0.76       0.77         38

 accuracy          0.75
 macro avg         0.73
 weighted avg      0.75
```



In order to find the best possible features for the prediction of status, I used the correlation technique which tells the importance of each feature with the target variable, the values vary between -1, 0 and 1, where -1 means that features have negative liner relationship with the target variable, 0 means that the feature has no liner relationship with the target variable and 1 mean the relationship is liner.

```
correlation_with_response
Bilirubin      0.441052
Prothrombin    0.402524
N_Days         0.394911
Copper         0.365267
Edema          0.324882
Albumin        0.309032
Ascites        0.297342
Stage          0.289185
Hepatomegaly   0.288782
SGOT           0.284466
Sex            0.213317
Alk_Phos       0.211104
Tryglicerides  0.207330
Platelets      0.177853
Age            0.175284
Spiders        0.162019
Cholesterol     0.148021
Drug           0.058632
Name: Status, dtype: float64
```

According to this, drug seems to be the feature with least linear relationship with the status, the rest of the features have little to maximum positive linear relationship with status.